

猫狗分类实验：方法、结果与分析

EE6483 项目参考材料 | 更新日期：2026-10-07。本报告整理五组已完成的核心实验，供组员核对结果和撰写正式报告。所有指标来自保存的实验记录，没有新增训练。

1. 我们完成了什么，主要发现是什么

我们使用课程猫狗数据，完成了预训练ResNet-18基线、数据增强、两个学习率以及MobileNetV3-Small对照，并为每组模型生成500条候选测试预测。此次训练只更新分类层，没有解冻骨干，也没有从头训练CNN。

ResNet-18基线验证准确率为98.80%。五组中，无增强、学习率0.0003取得最高验证准确率98.88%；MobileNetV3-Small当前配置为98.00%。增强在学习率0.001时净减少2张错误，但没有在两种学习率下表现出一致收益。较小学习率也并非总是更好。

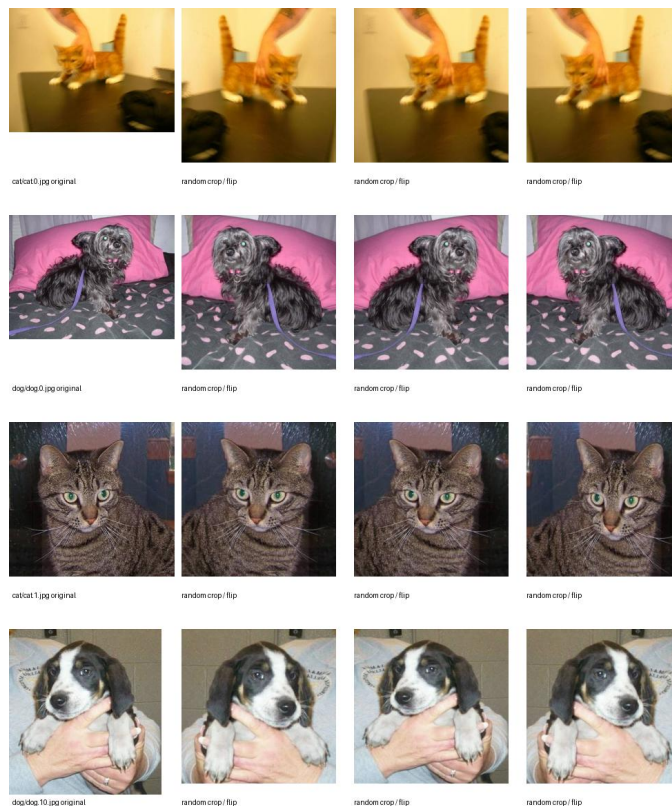
这些结果说明当前迁移学习方案能够较好完成验证集分类，但不能证明参数全局最优或增强收益稳定。同一验证集参与模型选择，结果可能偏乐观；无标签测试集的实际准确率仍未知。

2. 数据如何检查和处理

课程提供训练20,000张、验证5,000张、无标签测试500张，标签cat=0、dog=1。图片均可读取。哈希检查发现训练与验证有一对完全相同图片，按已确认规则，仅从训练清单排除cat.2339.jpg；原始文件和训练内部重复均保留。实际训练19,999张：猫9,999、狗10,000；验证每类2,500张。可读性和哈希检查不能排除近似重复、标签问题或分布差异。

训练集用于更新分类层，验证集用于选最佳轮次和比较配置，测试集用于最终预测。验证集是老师提供的图片，不是模型生成的数据。CIFAR-10属于后续独立任务，不用于生成猫狗提交文件。

E1/E3/E4将RGB图片短边缩放至256，再中心裁剪 224×224 ，最后采用ImageNet归一化。E2/E5训练改用随机裁剪：面积采样0.8 – 1.0，宽高比0.9 – 1.1，并以0.5概率水平翻转。所有验证和测试使用相同中心裁剪处理。随机裁剪用于增加训练输入变化，但也可能删去主体；极端宽高比下的回退裁剪不一定满足采样面积范围。



3. 为什么选择这两个模型，如何训练

ResNet-18使用ImageNet预训练权重，冻结骨干参数和BatchNorm统计，将512维特征连接到新建的512→2分类层，仅训练1,026个参数。选择它是为了利用已有视觉特征，在有限算力和时间下建立可复现基线。冻结骨干降低训练成本，也限制特征适应新任务的能力。

MobileNetV3-Small提供轻量模型方案。保留并冻结特征提取部分及原分类器前部，Dropout保持评估状态，只训练1024→2末层，共2,050个参数。它与ResNet-18不仅架构不同，保留分类器结构也不同，因此比较的是两套具体迁移学习方案，不能单独归因于架构。

两模型输入 $3 \times 224 \times 224$ ，输出两个类别分数。训练使用交叉熵损失，预测取较大分数对应类别。所有实验采用Adam、batch size 32、10轮、seed=42；每轮约625次更新。批大小是兼顾显存和效率的起始设置，10轮是统一预算，均未证明最优。

学习率0.001为起始值，0.0003用于检验较小更新步长。固定种子便于复现和控制部分随机因素，仍会发生洗牌与随机增强；只有一次运行不能评估随机波动。代码设置Python、NumPy、PyTorch及CUDA种子和cuDNN确定性选项，但不能保证跨硬件和版本完全一致。最佳模型按验证准确率选择，相同时优先较低验证损失。

E1/E2/E5逐批读取原图训练；E3/E4缓存确定性冻结特征后训练分类层，避免重复特征计算。因此耗时不能直接作为模型速度比较。历史缓存lr=0.001技术对照已按要求移除，本报告以保留的五组结果为依据，不将其列为核心实验。

4. 五组实验得到什么结果

编号	配置	最佳轮	验证准确率	验证损失	错误数
E1	ResNet-18, 无增强, lr=0.001	2	98.80%	0.035551	60
E2	ResNet-18, 有增强, lr=0.001	9	98.84%	0.033950	58
E3	ResNet-18, 无增强, lr=0.0003	10	98.88%	0.032862	56
E4	MobileNetV3-Small, 无增强, lr=0.001	1	98.00%	0.050044	100
E5	ResNet-18, 有增强, lr=0.0003	8	98.80%	0.033714	60

表中均为5,000张有标签验证图片的结果，不是测试准确率。E3正确4,944张、错误56张，成为当前最高验证准确率候选，但最终submission.csv尚未确定。

5. 分别比较增强、学习率和模型的影响

先看增强：E1到E2准确率98.80%→98.84%，修正16张、新增14张、持续错误44张，净少错2张。猫召回99.04%→98.92%，狗召回98.56%→98.76%。这次小幅整体改善伴随类别取舍，不能写成两类都变好。

再看无增强学习率：E1到E3准确率98.80%→98.88%，净少错4张。猫召回99.04%→98.56%，狗召回98.56%→99.20%。同时存在原图与缓存执行方式差异，需要保留这一限制，不能将变化完全归因于学习率。E3最好结果在第10轮，未继续训练，无法判断延长预算是否改善。

增强条件下，E2到E5仅修改学习率，准确率98.84%→98.80%，修正11张、新增13张、持续错误47张。猫召回98.92%→98.48%，狗召回98.76%→99.12%。两类训练数量仅相差1张，不能用“狗更多”解释这种变化。优化过程可能改变分类边界，但具体原因尚未验证。

E5验证损失从0.033950降至0.033714，而准确率下降。准确率只统计类别是否正确，交叉熵还考虑预测分数；两者不必同步。按照预先确定的准确率优先规则，增强配置中仍优先E2。

最后看模型：E4达到98.00%，低于当前ResNet-18各配置。它并未进行独立参数调优，因此只能说当前MobileNet方案表现较低，不能推断轻量模型普遍较差。

6. 每类指标和混淆矩阵如何阅读

召回率表示某类真实图片有多少被正确识别；精确率表示预测为某类的图片有多少正确；F1综合两者。混淆矩阵的行是真实cat/dog，列是预测cat/dog。下面保留各组完整类别结果，便于核对整体改善是否牺牲某一类。

E1：ResNet-18，无增强，lr=0.001

类别	Precision	Recall	F1
cat	98.57%	99.04%	98.80%
dog	99.04%	98.56%	98.80%

真实类别	预测猫	预测狗
猫	2476	24
狗	36	2464

E2：ResNet-18，有增强，lr=0.001

类别	Precision	Recall	F1
cat	98.76%	98.92%	98.84%
dog	98.92%	98.76%	98.84%

真实类别	预测猫	预测狗
猫	2473	27
狗	31	2469

E3：ResNet-18，无增强，lr=0.0003

类别	Precision	Recall	F1
cat	99.19%	98.56%	98.88%
dog	98.57%	99.20%	98.88%

真实类别	预测猫	预测狗
猫	2464	36
狗	20	2480

E4：MobileNetV3-Small，无增强，lr=0.001

类别	Precision	Recall	F1
cat	98.23%	97.76%	98.00%
dog	97.77%	98.24%	98.00%

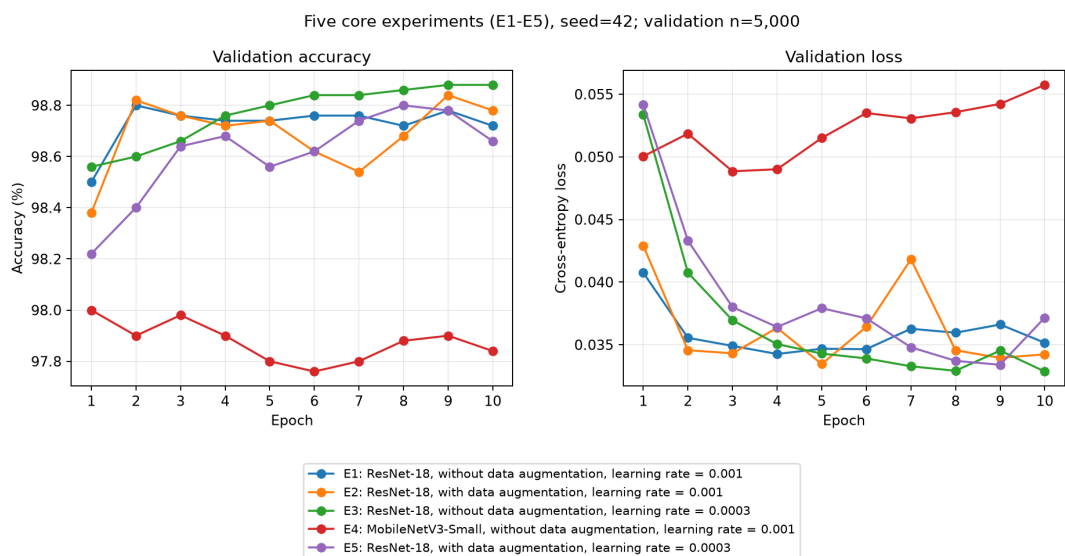
真实类别	预测猫	预测狗
猫	2444	56
狗	44	2456

E5：ResNet-18，有增强，lr=0.0003

类别	Precision	Recall	F1
cat	99.11%	98.48%	98.80%
dog	98.49%	99.12%	98.80%

真实类别	预测猫	预测狗
猫	2462	38
狗	22	2478

7. 曲线告诉我们什么



曲线用于观察10轮内的变化，不代表已经搜索到最佳训练轮数。E1最佳在第2轮、E2第9轮、E3第10轮、E4第1轮、E5第8轮。继续降低训练损失不一定改善验证表现；增强训练准确率是在变化的输入上统计，不能直接与无增强训练准确率等同。

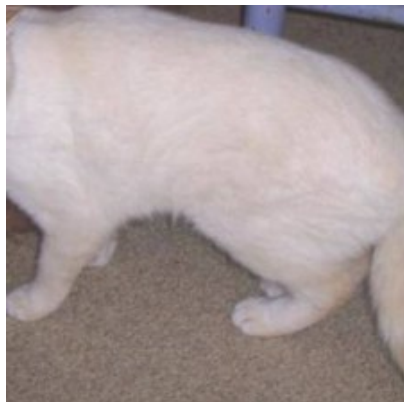
8. 从正确和错误图片中能观察到什么

下面保留实际原图和预处理图。案例用于说明预测变化和处理后的可见内容，不用于证明因果。背景、遮挡或裁剪可能影响识别，但需要进一步对照才能确认。

修正错误: cat.10919.jpg



原图（保持比例）



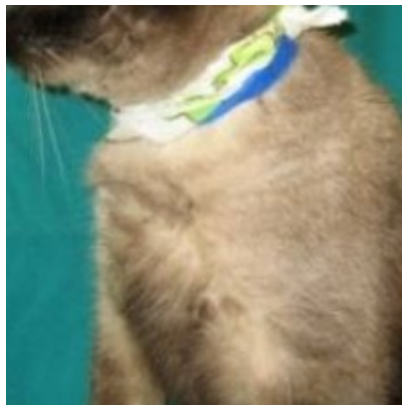
实际验证预处理后 RGB 图

课程标签 0；D1 1；D2 0。0=猫，1=狗。按清单前两项确定性选取，不能仅凭预测变化证明某个增强机制起效。

修正错误: cat.1799.jpg



原图（保持比例）



实际验证预处理后 RGB 图

课程标签 0；D1 1；D2 0。0=猫，1=狗。按清单前两项确定性选取，不能仅凭预测变化证明某个增强机制起效。

新增错误: cat.10570.jpg



原图（保持比例）



实际验证预处理后 RGB 图

课程标签 0；D1 0；D2 1。0=猫，1=狗。按清单前两项确定性选取，不能仅凭预测变化证明某个增强机制起效。

新增错误: cat.10912.jpg



原图（保持比例）



实际验证预处理后 RGB 图

课程标签 0；D1 0；D2 1。0=猫，1=狗。按清单前两项确定性选取，不能仅凭预测变化证明某个增强机制起效。

持续错误: cat.10129.jpg



原图（保持比例）



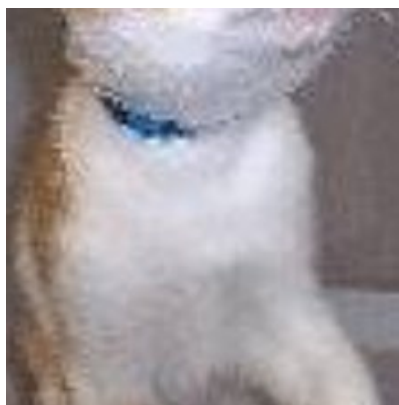
实际验证预处理后 RGB 图

课程标签 0；D1 1；D2 1。0=猫，1=狗。按清单前两项确定性选取，不能仅凭预测变化证明某个增强机制起效。

持续错误: cat.10418.jpg



原图（保持比例）



实际验证预处理后 RGB 图

课程标签 0；D1 1；D2 1。0=猫，1=狗。按清单前两项确定性选取，不能仅凭预测变化证明某个增强机制起效。

两模型均正确: cat.100.jpg



原图（保持比例）



实际验证预处理后 RGB 图

课程标签 0；D1 0；D2 0。0=猫，1=狗。按清单前两项确定性选取，不能仅凭预测变化证明某个增强机制起效。

两模型均正确: dog.1.jpg

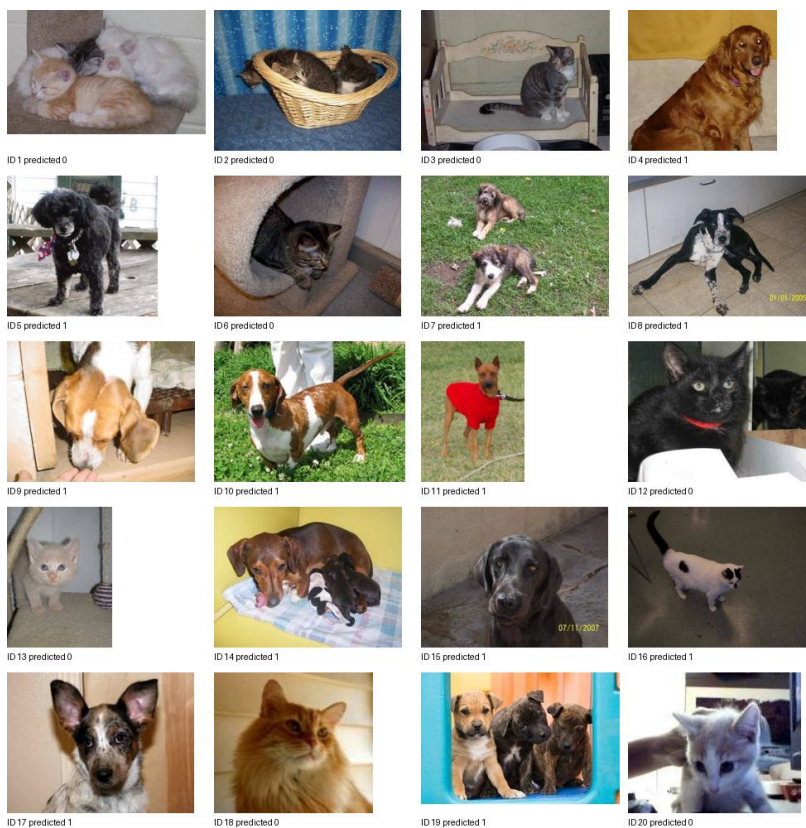


原图（保持比例）



实际验证预处理后 RGB 图

课程标签 1；D1 1；D2 1。0=猫，1=狗。按清单前两项确定性选取，不能仅凭预测变化证明某个增强机制起效。



图：D2测试前20张及真实输出，用ID4和ID16作人工观察案例。

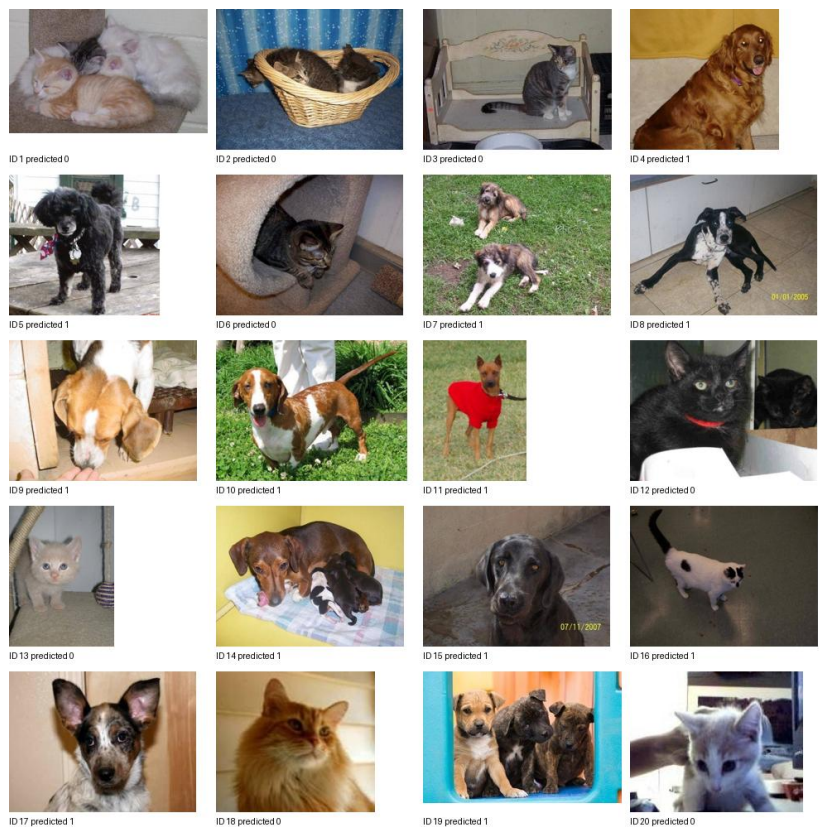
具体案例观察：首个修正例为户外猫，另一修正例为狗与人物同框；模型可以在有人物和背景的图片中正确分类，但不能据此宣称背景鲁棒性提高。新增错误包括侧身卧猫及佩戴项圈的猫，均是原D2已修正过的图片，表明改变学习率可以撤销先前个例收益。两例持续错误仍是猫的普通照片，增强与较小学习率组合没有消除这些失败。原图／裁剪对照可见部分上下文被删除，这是可观察的预处理影响；没有替代裁剪对照，不能认定它就是错误的原因。

cat.10919和cat.1799被E2修正，但E5又判错，说明改变学习率可能撤销个例收益。cat.10129和cat.10418在E1/E2/E5中仍被判错；这些普通照片也提醒我们，高整体准确率不意味着模型能处理所有正常图片。

9. 500条预测如何使用，测试效果能否确定

五组均生成id,label候选CSV，500条ID为1 – 500，cat=0、dog=1。它们来自课程猫狗test目录，与验证预测和CIFAR-10不同。最终需选定一个模型，将对应该候选整理为唯一submission.csv；当前未自动选择。

测试ID4视觉上为狗且E2预测狗；ID16视觉上为猫但E2预测狗。这是人工观察，不是官方标签评价，不能用于计算测试准确率或认定官方错标。保留模型直接输出，不根据人工判断修改候选标签。



图：D2测试前20张及真实输出，用ID4和ID16作人工观察案例。

10. 当前结论、完成范围与下一步

猫狗部分已覆盖数据检查、模型结构、训练策略、两个学习率尝试、模型及增强比较、验证指标、案例和候选预测，可用于撰写主要实验章节。文献综述、三人贡献、正式引用、最终提交选择仍需完善；CIFAR-10和不均衡实验尚未执行，不能将整个课程项目写成已完成。

目前结果只来自固定划分、单一种子和有限配置。同一验证集同时用于模型选择和汇报表现，可能偏乐观。98.88%是已测试配置的最高成绩，不能称全局最优；增强收益尚未证明稳定。训练已接近完整20,000张，没有证据认定样本少是主要问题。

优先完成CIFAR-10十分类和不均衡任务。若有额外时间，先围绕增强是否可重复，在预先固定的另外两个种子下成对比较E1/E2；若目标是提高识别能力，再单独设计冻结与局部解冻对照。两者解决不同问题，目前均未执行。本次修改不启动训练。

附录A：每轮真实训练记录

E1：ResNet-18，无增强，lr=0.001

轮次	训练损失	训练准确率	验证损失	验证准确率
1	0.068835	97.50%	0.040751	98.50%
2	0.037406	98.64%	0.035551	98.80%
3	0.033546	98.77%	0.034906	98.76%
4	0.032004	98.84%	0.034261	98.74%
5	0.030255	98.88%	0.034679	98.74%
6	0.029998	98.92%	0.034646	98.76%
7	0.028844	98.95%	0.036275	98.76%
8	0.027837	98.95%	0.035959	98.72%
9	0.028016	99.00%	0.036614	98.78%
10	0.027760	98.95%	0.035159	98.72%

E2：ResNet-18，有增强，lr=0.001

轮次	训练损失	训练准确率	验证损失	验证准确率
1	0.066378	97.63%	0.042906	98.38%
2	0.036367	98.64%	0.034563	98.82%
3	0.033366	98.77%	0.034313	98.76%
4	0.032328	98.80%	0.036344	98.72%
5	0.031740	98.79%	0.033456	98.74%
6	0.031366	98.82%	0.036427	98.62%
7	0.028756	98.94%	0.041821	98.54%
8	0.030314	98.94%	0.034555	98.68%
9	0.028542	98.95%	0.033950	98.84%
10	0.028789	98.96%	0.034231	98.78%

E3：ResNet-18，无增强，lr=0.0003

轮次	训练损失	训练准确率	验证损失	验证准确率
1	0.125224	95.71%	0.053375	98.56%
2	0.047005	98.46%	0.040753	98.60%
3	0.039464	98.64%	0.036951	98.66%
4	0.036046	98.79%	0.035047	98.76%
5	0.033895	98.76%	0.034294	98.80%
6	0.032615	98.83%	0.033906	98.84%
7	0.031478	98.79%	0.033278	98.84%
8	0.030473	98.88%	0.032920	98.86%
9	0.029946	98.92%	0.034533	98.88%
10	0.029195	98.95%	0.032862	98.88%

E4：MobileNetV3-Small，无增强，lr=0.001

轮次	训练损失	训练准确率	验证损失	验证准确率
1	0.069474	97.59%	0.050044	98.00%
2	0.046251	98.32%	0.051852	97.90%
3	0.041308	98.40%	0.048846	97.98%
4	0.038245	98.65%	0.049009	97.90%
5	0.035757	98.63%	0.051504	97.80%
6	0.033789	98.76%	0.053498	97.76%
7	0.032428	98.86%	0.053074	97.80%
8	0.031108	98.83%	0.053566	97.88%
9	0.030147	98.91%	0.054231	97.90%
10	0.028674	98.93%	0.055746	97.84%

E5：ResNet-18，有增强，lr=0.0003

轮次	训练损失	训练准确率	验证损失	验证准确率
1	0.123170	95.75%	0.054164	98.22%
2	0.046387	98.47%	0.043312	98.40%
3	0.038189	98.72%	0.038003	98.64%
4	0.035685	98.70%	0.036417	98.68%
5	0.034218	98.75%	0.037915	98.56%
6	0.033486	98.71%	0.037108	98.62%
7	0.030768	98.86%	0.034796	98.74%
8	0.031715	98.84%	0.033714	98.80%
9	0.029373	98.98%	0.033399	98.78%
10	0.029853	98.89%	0.037159	98.66%

附录B：代码与复现材料

模型定义见src/models/resnet18.py；共享数据、预处理、种子和训练循环见src/common/；阶段入口见src/phase1_catdog_baseline/和src/phase2_catdog_improvements/。项目README提供运行说明；运行前需配置数据路径和Python环境。已有运行名不可直接重复覆盖。

src/models/resnet18.py

```
"""Frozen ImageNet ResNet-18 baseline; keep BatchNorm statistics fixed."""
from torch import nn
from torchvision.models import resnet18, ResNet18_Weights

def build_model(pretrained=True):
    weights = ResNet18_Weights.IMAGENET1K_V1 if pretrained else None
    model = resnet18(weights=weights)
    for parameter in model.parameters():
        parameter.requires_grad = False
    model.fc = nn.Linear(model.fc.in_features, 2)
    return model

def baseline_transform():
    return ResNet18_Weights.IMAGENET1K_V1.transforms()
```

src/common/utils.py

```
"""Reproducible setup and portable JSON/checkpoint metadata."""
import json
import random
from pathlib import Path
import numpy as np
import torch

ROOT = Path(__file__).resolve().parents[2]

def seed_everything(seed):
    random.seed(seed)
    np.random.seed(seed)
    torch.manual_seed(seed)
    torch.cuda.manual_seed_all(seed)
    torch.backends.cudnn.benchmark = False
```

```
torch.backends.cudnn.deterministic = True
```

```
def save_json(path, value):
    path = Path(path)
    path.parent.mkdir(parents=True, exist_ok=True)
    path.write_text(json.dumps(value, ensure_ascii=False, indent=2), encoding="utf-8")
```

```
def load_checkpoint(path, device):
    from src.models.resnet18 import build_model
    checkpoint = torch.load(path, map_location=device, weights_only=True)
    if checkpoint["label_mapping"] != {"cat": 0, "dog": 1}:
        raise ValueError("Unexpected checkpoint label mapping")
    model = build_model(pretrained=False).to(device)
    model.load_state_dict(checkpoint["model_state"])
    return model, checkpoint
```

src/common/transforms.py

```
"""D2 training augmentation; validation always uses the weight preset."""
```

```
from torchvision import transforms
from torchvision.transforms import InterpolationMode
```

```
def augmented_transform(config):
    return transforms.Compose([
        transforms.RandomResizedCrop(224, scale=tuple(config["crop_scale"]),
                                          ratio=tuple(config["crop_ratio"]),
                                          interpolation=InterpolationMode.BILINEAR, antialias=True),
        transforms.RandomHorizontalFlip(p=config["horizontal_flip_probability"]),
        transforms.ToTensor(),
        transforms.Normalize(mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]),
    ])
```

补充实验的实际分类头优化与选模片段

```
optimizer=torch.optim.Adam(head.parameters(),lr=lr)
generator=torch.Generator().manual_seed(42)
loader=DataLoader(TensorDataset(*features['train']),batch_size=32,shuffle=True,generator=generator)
best=(-1,float('-inf'));history=[]
for epoch in range(1,11):
    started=time.perf_counter();loss_sum=correct=0;head.train()
    for x,y in loader:
        x,y=x.to(device),y.to(device);optimizer.zero_grad(set_to_none=True);logits=head(x);loss=nn.functional.cross_entropy(logits,y);loss.backward();optimizer.step()
        loss_sum+=loss.item()*len(y);correct+=(logits.argmax(1)==y).sum().item()
    head.eval()
    with torch.inference_mode():
        vx,vy=features['val'];logits=head(vx.to(device));loss=nn.functional.cross_entropy(logits,vy.to(device)).item();pred=logits.argmax(1).cpu();acc=(pred==vy).float().mean().item()
        # Compute exact fraction for direct comparison.
        acc=int((pred==vy).sum())/len(vy)
        record={'epoch':epoch,'train_loss':loss_sum/len(data['train']),'train_accuracy':correct/len(data['train']),'val_loss':loss,'val_accuracy':acc,'seconds':time.perf_counter()-started};history.append(record)
        key=(acc,-loss)
        if key>best:
            best=key
            torch.save({'backbone_state':model.cpu().state_dict(),'head_state':head.cpu().state_dict(),'config':config,'epoch':epoch,'label_mapping':LABELS},out/'best.pth');model.to(device);head.to(device)
            predictions=pred.tolist();metrics={'epoch':epoch,'accuracy':acc,'loss':loss,'count':len(vy),'confusion_matrix':confusion_matrix(vy,pred,labels=[0,1]).tolist(),'classification_report':classification_report(vy,pred,labels=[0,1],target_names=['cat','dog'],output_dict=True,zero_division=0)}
```

```
save_json(result/'metrics.json',metrics);save_json(out/'best_metrics.json',metrics)
```

```
save_json(result/'predictions.json',[{ 'path':str(p.relative_to(ROOT/'datasets')), 'label':y, 'prediction':z} for  
(p,y),z in zip(data['val'].samples,predictions)])
```

实现依据：ResNet论文He等，Deep Residual Learning for Image Recognition，CVPR 2016；MobileNetV3论文Howard等，Searching for MobileNetV3，ICCV 2019。此处只说明模型来源，不替代后续完整文献综述。

附录C：结果索引及来源校验

下列路径相对项目根目录。各组文件存放在自己的结果目录，不在综合目录重复复制。runs存放本地日志和权重且被Git忽略；组员核对结果以results/README.md为入口。

E1：results/phase1_catdog_baseline/baseline_full_20261006_001

config.json SHA-256：08faa69535a4974eb7ac5c7e66eee21a6fff249778f6916bc5e996532074f6a2

history.csv SHA-256：a8465fc0ed5992cb5c50f4d4f12ab6def3b6091e4d45998034d32612a5028dc3

metrics.json SHA-256：6e686f43c53a35a2eb2c4ae360ac06535771686fb178634033bf801c6dd5dfb3

predictions.json SHA-256：93e188a6b9ea3f59b0134570c3dea3a341de77185633f2ef456ce8bed0137c61

E2：results/phase2_catdog_improvements/augmentation_full_20261006_001

config.json SHA-256：1f50b444b1c996e1d223f65684db9a118c532557ffcb81e01002eca60f4f618

history.csv SHA-256：3c3b6958f01d7d9ecbf608a629da4e4a9cc8b664e71ac90fde3d31f5b2eb58f0

metrics.json SHA-256：c601dbb5d41d4fc797e5247f3210bee4eebe11008a2766e8ffbf925f8ce53a0

predictions.json SHA-256：eaf06e2c48e8bdaa54a8d7b0f8f756841d3f8e029cd1f66425aa7de5d770f97b

E3：results/phase2_catdog_improvements/resnet18_lr0003

config.json SHA-256：2dfc90becc6fce1847b9cf8755a2777204cd6d71a5c889db876e1cc60589cd19

history.csv SHA-256：4ebc42ba560d1a4bdbb054738a3cebd49f8f6840268e0e6ab3e47a3c44f36ed3

metrics.json SHA-256：d0c1f03672bc47202e1ac530eae817a5133e9cd94c00ab955f30a1f862a490b1

predictions.json SHA-256：634124d75c09cbec611209bd439bd02f1c78b71479e0dc7b3bfa5360406c85ab

E4：results/phase2_catdog_improvements/mobilenetv3small_baseline

config.json SHA-256：f2b80d68a7fd5a1f9f5ce9103e800cc2a6435598ca2be3fda4229623e1b41a9c

history.csv SHA-256：d930b9eb0b9c8fa39d39c7ba38f08e8db89dabfa4580c2d96d5ae2c9cf9ca221

metrics.json SHA-256：cf51b5436e4c7aaac8ef31a09df1c9d98797f4e4021fb807dceaa01da7c7028d

predictions.json SHA-256：aa697a3cefd016c467ffc23f15087567e3287dd69d3a8702a5573162fd7a3a80

E5：results/phase2_catdog_improvements/augmentation_lr0003_20261006_001

config.json SHA-256：a6b873c362cfb1dad850d86a59a32c3c320d2ca7cafbabbafc57cb8031f2b501

history.csv SHA-256：15f4ce7709a01275ae11cc557c3f0975b6176a815088fa6b6d13a9036ac563c5

metrics.json SHA-256：385c76b6e08699d992a5400531901255806cbcea87b146356c3e65564187fda3

predictions.json SHA-256：7a5fbf74b53301752ccb40b5436838504f24495489692298dd21f157c02ee74d